

DSA Day 29: Huffman & B-Trees

Book: Data Structures Through C in Depth - Ch6 §6.18-6.21 · pp 287-318 | Code in Python

Overview

Huffman builds an optimal prefix code by greedily merging the two lowest-frequency nodes. B-trees are balanced multi-way trees for disk indexing.

Key concepts to master

- Greedy merge (Huffman)
- Prefix codes
- B-tree order, splits & merges

Python implementation

```
import heapq
def huffman(freq):
    h=[[w,[c,'']] for c,w in freq.items()]; heapq.heapify(h)
    while len(h)>1:
        lo=heapq.heappop(h); hi=heapq.heappop(h)
        for p in lo[1:]: p[1]='0'+p[1]
        for p in hi[1:]: p[1]='1'+p[1]
        heapq.heappush(h,[lo[0]+hi[0]]+lo[1:]+hi[1:])
    return h[0][1:]
```

Complexity

Huffman $O(n \log n)$; B-tree ops $O(\log n)$.

Common pitfalls

- Non-prefix codes
- Not splitting a full B-tree node

Practice

- Build Huffman codes for a short text
- Why do databases use B-trees?

